

# MID

Paper: MID - Set A | Duration: 2 Hours | Max Marks: 60 | Date: 14-03-2026

---

Total Questions: 11 | Answer all questions.

Name: \_\_\_\_\_

Roll No: \_\_\_\_\_

Student Sign: \_\_\_\_\_

Teacher Sign: \_\_\_\_\_

---

## SECTION A: MCQS

1. Medium MCQ Which of the following problems in Lexi is explicitly solved using the Abstract Factory Pattern? [5 Marks]

- a) How to internally represent the document's physical hierarchy.
- b) How to arrange text and graphics into lines and columns.
- c) How to create UI widgets specific to a particular look-and-feel without hard-coding class names.
- d) How to provide a uniform mechanism for user commands and support undo/redo functionality.

2. Hard MCQ In Lexi's implementation of the Bridge Pattern for supporting multiple window systems, what is the primary role of the WindowImp hierarchy? [5 Marks]

- a) To define the high-level interface for application windows.
- b) To encapsulate the formatting algorithm for line-breaking.
- c) To define the low-level, platform-specific operations for windowing.
- d) To represent the document's hierarchical structure of objects.

3. Medium MCQ The problem of 'Embellishing the UI' in Lexi is solved by having embellishments like Border and Scroller be subclasses of Glyph that enclose a single component and forward requests. This design is a direct application of which pattern? [5 Marks]

- a) Composite Pattern
- b) Strategy Pattern
- c) Decorator Pattern
- d) Abstract Factory Pattern

4. Hard MCQ According to the source text, what specific design pattern is used to encapsulate the complex algorithm for breaking text into lines and columns, allowing it to be easily swapped for different trade-offs? [5 Marks]

- a) Composite Pattern, by treating all elements uniformly.
- b) Strategy Pattern, by encapsulating the algorithm in a separate class hierarchy.
- c) Command Pattern, by encapsulating user operations as objects.
- d) Decorator Pattern, by adding responsibilities to an object transparently.

5. Medium MCQ When addressing the 'Document Structure' problem, Lexi uses a common abstract class Glyph. What is the primary purpose of this Glyph class in conjunction with the Composite Pattern? [5 Marks]

- a) To encapsulate formatting algorithms for text and graphics.
- b) To define a uniform interface for all document elements, primitive or structural.
- c) To provide a mechanism for dynamically adding visual embellishments.
- d) To separate the windowing abstraction from its platform-specific implementation.

**6. Hard MCQ** Lexi's GUIFactory abstract class, which defines interfaces like CreateScrollBar and CreateButton, is central to supporting multiple look-and-feel standards. What is the key benefit of this approach as described in the Mark?

- a) It allows for dynamically adding new visual elements to the UI at runtime.
- b) It encapsulates the knowledge of which concrete widget classes to use, decoupling application code from a particular platform.
- c) It enables traversal of the document's glyph structure without modifying glyph classes.
- d) It provides a uniform mechanism for handling user commands and supporting undo/redo.

## SECTION A: SHORT ANSWERS

**7. Medium Short Answer** Explain how Lexi's solution for 'Document Structure' leverages both recursive composition and the Composite Pattern to treat primitive and structural elements uniformly.

The solution utilizes a hierarchical tree structure where all elements, from primitive characters to structural rows, are subclasses of a common abstract Glyph class. This recursive composition, combined with the Composite Pattern, allows clients to interact with individual Glyph objects and compositions of Glyph objects (like a row containing characters and other compositions) through the same simplified interface. This uniformity eliminates the need for complex conditional logic, as all elements are treated as Glyphs.

[6 Marks]

**8. Medium Short Answer** The Formatting problem in Lexi aims to allow different trade-offs between speed and quality for line-breaking algorithms. Describe how the Strategy Pattern facilitates this flexibility without altering the document's core structure.

The Strategy Pattern facilitates this by encapsulating the formatting algorithm in a separate class hierarchy, rooted by a Compositor class. A Composition glyph class then holds a reference to a Compositor object and delegates the formatting work to it. This decouples the document's structure from the algorithm, allowing different formatting algorithms (strategies) to be easily swapped at runtime. New strategies can be introduced without modifying the core document structure classes, ensuring flexibility in speed-quality trade-offs.

[6 Marks]

**9. Hard Short Answer** Lexi addresses the challenge of dynamically adding visual embellishments without a "combinatorial explosion" of subclasses. Elaborate on how the Decorator Pattern achieves this by integrating embellishments with the Glyph interface.

The Decorator Pattern solves this by making embellishments themselves subclasses of the Glyph abstract class (e.g., Border, Scroller). These decorator Glyph objects enclose a single child component, also a Glyph, and implement the same Glyph interface. They transparently forward most requests to their enclosed child but add their own behavior (like drawing a border) before or after the forwarding. This allows responsibilities to be added to an object dynamically and recursively wrapping any Glyph with multiple decorators, avoiding a large number of distinct subclasses for every possible combination of features.

[6 Marks]

**10. Medium Short Answer** For user operations in Lexi, supporting multi-level undo/redo is crucial. Explain how the Command Pattern, including its Execute and Unexecute methods, along with a history list, enables this functionality.

The Command Pattern encapsulates each user operation as an object from a Command class hierarchy. Each Command object has an Execute method which performs the operation when activated by UI elements like menu items or buttons. To support undo/redo, commands can also implement an Unexecute method, which reverses the operation. A command history list manages the sequence of executed commands, storing them. This list can then be traversed backwards using Unexecute for undo and forwards using Execute for redo, providing robust multi-level undo/redo functionality.

[6 Marks]

**11.** Hard Short Answer Lexi utilizes both the Iterator and Visitor Patterns for analytical operations like spelling checking and hyphenation. Discuss the distinct role each pattern plays in solving this problem without modifying glyph classes or using type-checking.

The Iterator Pattern abstracts the process of traversing the document's glyph structure, allowing different traversal methods (e.g., pre-order, in-order) to be implemented without changing the glyph classes themselves. It provides a standardized way to access elements sequentially.

The Visitor Pattern encapsulates the analysis logic. A Visitor object is passed to each glyph during a traversal. Each glyph "accepts" the visitor by calling a type-specific method on it (e.g., VisitCharacter, VisitRow). This allows the analyzer to perform glyph-specific operations without requiring type-checking within the analyzer and without adding analytical methods directly to the Glyph classes, keeping them focused on structure.

**[6 Marks]**

---

**--- End of Question Paper ---**